

Reineke-RAG

Technical Description

Reineke-RAG — 2026

Reineke-RAG — Technical Description

An expert-level technical description of what this repository contains today, what it is designed to become, and how the two are connected.

1. What this repository is (today)

Reineke-RAG is — **at this point in its lifecycle** — a **blueprint repository**: concept, architecture, decision records, phased implementation plan, subagent briefs, and a skeleton Docker Compose + Makefile + `.env` template. **No application code ships in the repo yet.** The build is produced by an AI *coordinator agent* that dispatches work to seven specialist subagents defined in [docs/06_AGENT_BRIEFS.md](#).

The repository is frozen on **two contracts**:

1. **Concept + ADRs are immutable** without a new ADR superseding them. See [CLAUDE.md](#) § "House rules".
2. **Ownership boundaries** between subagents are strict; every file has exactly one owner.

This gives the build reproducibility: any new coordinator started on the same repo sees the same plan, the same acceptance criteria, the same handoff points.

1.1 Repository layout

/	
├─ README.md	orientation
├─ CLAUDE.md	project instructions for AI agents
├─ Makefile	top-level targets (bootstrap, up, pull-models...)
├─ BUILD_LOG.md	append-only record of build events (coordinator-maint)
├─ HANDOVER.md	sign-off template; populated at Phase 10
├─ .gitignore	
├─ .claude/	
│ └─ settings.local.json	
│ └─ agents/	eight subagent definitions (.md with frontmatter)
├─ agents/	(empty; reserved)
├─ config/	
│ └─ .env.example	full environment template
│ └─ docker-compose.yml	skeleton Compose (25 services, pinned tags, 2 profile)
│ └─ owner-inputs.yaml.example	owner-facing Phase-0 inputs
├─ docs/	
│ └─ 01_CONCEPT.md	north-star concept, requirements, tech decisions
│ └─ 02_ARCHITECTURE.md	components, flows, schemas, interfaces
│ └─ 03_HANDBOOK.md	end-user handbook (target behaviour)
│ └─ 04_OPERATIONS.md	admin/IT handbook (target behaviour)
│ └─ 05_IMPLEMENTATION_PLAN.md	phased build plan with acceptance criteria
│ └─ 06_AGENT_BRIEFS.md	coordinator + 7 specialist subagent specs
└─ adr/	eight accepted Architecture Decision Records

1.2 What a reader can currently do

- Read the full concept and architecture, end-to-end.
 - Inspect every pinned container image and its role.
 - Reproduce the build by starting the `coordinator` subagent with `config/owner-inputs.yaml` populated.
 - Review the decision rationale for every significant technical choice (eight ADRs).
 - **Not** run `make up` — the concrete service code (`services/**`) is produced during build, not seeded.
-

2. The target system

When fully built (Phase 10 countersigned), Reineke-RAG is a **fully offline, enterprise-grade Retrieval-Augmented Generation stack** for internal Word / PDF / Excel documents in a mixed German + English corpus. Single-host deployment by Docker Compose, designed for an Apple M4 Max 64 GB reference box or Linux + ≥ 24 GB GPU.

2.1 Core capabilities

1. **Ingests PDF, DOCX, XLSX** — including scanned PDFs (OCR), multi-column layouts, embedded tables, formulas, multi-sheet spreadsheets.
2. **Answers four query classes:** `lookup`, `extraction`, `table-math`, `synthesis` — each routed to a different LLM tier.
3. **Bilingual DE + EN** first-class; embedder and LLMs all strong in both.
4. **Every answer cites.** The system prompt refuses when no retrieved chunk supports a claim. Citations point to the original file, page, and section.
5. **ACL-aware.** Every Qdrant/DuckDB query carries a mandatory group filter; the filter is an assertion, not a configuration flag.
6. **Structure-aware retrieval.** Tables survive ingestion as typed columnar data in DuckDB *in addition* to being embedded. Table-math queries run real SQL.
7. **Hybrid retrieval.** Dense + sparse in a single Qdrant call, fused by RRF, reranked by a cross-encoder.
8. **Offline contract.** No outbound calls at runtime — verified with pfSense / Little Snitch in Phase 9.
9. **Full trace per query** into Langfuse: `classify` → `rewrite` → `dense/sparse` → `rerank` → `SQL` → `generate`.

2.2 What it explicitly does NOT do (v1)

- No web browsing, no email/Slack ingestion, no agentic workflows beyond optional n8n.
- No HA or clustering (single-node + backups).
- No per-document fine-grained ACLs (folder-level only).
- No languages other than DE + EN in the evaluated matrix.
- No chat-attached file uploads (would bypass ACLs).

See [docs/01_CONCEPT.md](#) §10.

3. Technology stack

Everything is permissively licensed (Apache 2.0 / MIT / PostgreSQL / BSD-3).

Layer	Choice	Version / tag	Rationale (ADR)
Document parsing	Docling (IBM)	3 GB Python image	Best-in-class tables + layout; Apache 2.0 (ADR-001)
OCR	EasyOCR (default) / Tesseract deu+eng	in-image	Swappable via env
Embeddings (dense + sparse)	bge-m3	ollama pull bge-m3 , 1024-d	One model, two modes, DE+EN (ADR-003)
Vector DB	Qdrant	qdrant/qdrant:v1.12.0	Native hybrid + payload filters (ADR-002)
Reranker	BAAI/bge-reranker-v2-m3 via TEI	cpu-1.5 (default)	Multilingual cross-encoder, fast on Metal
LLM server	Ollama	ollama/ollama:0.3.12	Apple-Silicon native, lazy-load, OpenAI-compatible
LLMs (tiered)	Gemma 2 9B / Qwen 2.5 32B / Llama 3.3 70B	Q5/Q4 quants	Route by query class (ADR-004)
XLSX / tables	DuckDB	embedded, file-based	SQL path for numeric questions (ADR-006)
Object storage	MinIO	RELEASE.2024-08-03T...	Immutable raw files
Metadata	PostgreSQL 16	postgres:16-alpine	Users mirror, folders, docs, chunks, audit, jobs
Queue	Redis + RQ	redis:7-alpine	Simple ingestion queue
Identity	Authentik	2024.8	OIDC + groups + blueprints (ADR-005)
UI	Open WebUI + Pipelines	main	OIDC-ready, custom pipeline bridges to retrieval-api (ADR-007)
Observability (LLM)	Langfuse (self-hosted)	langfuse/langfuse:2.71	Span-per-step tracing
Observability (infra)	Prometheus v2.55.0 , Grafana 11.2.0 , Loki 2.9.10	provisioned JSON dashboards	Container + host metrics, logs
Reverse proxy	Caddy	caddy:2-alpine	Auto-HTTPS via internal CA
Orchestration (optional, profile automation)	n8n	1.60.0	Scheduled jobs, folder watcher

Not adopted as frameworks (see [ADR-008](#)): LangChain / LlamaIndex / Haystack. Their chunkers and SQL helpers may be imported as components behind feature flags; the glue is explicit FastAPI code.

4. Architecture at a glance

4.1 Component inventory (25 containers at max profile)

Category	Services
Edge	caddy
Identity	authentik-server, authentik-worker, authentik-db, authentik-redis
Storage	postgres, redis, minio, qdrant, duckdb-api (+ embedded DuckDB file)
LLM runtime	ollama, ollama-init (one-shot), tei-reranker
Custom services	docling, ingestion-api, ingestion-worker, retrieval-api, duckdb-api
UI	openwebui, pipelines
Observability	langfuse, langfuse-db, prometheus, grafana, loki, promtail
Automation (profile)	n8n, watcher

Host-bound ports: **only 80 and 443** (through Caddy). Everything else speaks over the `reineke` Docker bridge by service name.

4.2 Ingestion data flow

```
File drop → ingestion-api → MinIO (raw/{doc_id}/{filename})
                               → Postgres rag.documents (status=queued)
                               → Redis queue (RQ)

Worker ← Redis
        → Docling /parse   (DoclingDocument + tables)
        → HybridChunker    (structure-aware, max 512 tokens)
        → Ollama embed      (bge-m3 dense, 1024-d)
        → bge-m3 sparse     (in-process)
        → Qdrant upsert     (dense + sparse + payload incl. acl_groups)
        → DuckDB tables     (XLSX sheets, confident PDF tables)
        → Postgres rag.chunks / rag.tables
        → Postgres rag.documents (status=indexed)
```

Idempotent: `(folder_path, sha256)` dedupes; failure is transactional (no orphan points).

4.3 Retrieval + generation flow

Query in Open WebUI → Pipelines → retrieval-api (JWT forwarded)

```
retrieval-api:
  1. verify_jwt (JWKS cached)
  2. principal.groups → ACL filter
  3. classify(query) → lookup | extraction | table-math | synthesis [Gemma 9B]
  4. (optional) HyDE / paraphrase x2
  5. embed(q) dense + sparse
  6. Qdrant prefetch:
      dense (top 50, filter acl_groups ANY groups)
      sparse (top 50, same filter)
      fuse via RRF
      → top 50
  7. TEI rerank → top 12
  8. if table-math: LLM writes SQL → duckdb-api validates + runs on
      views.v_<table>_<group_hash> (ACL baked in)
  9. build prompt (DE/EN/bilingual), pick LLM tier
  10. Ollama stream → SSE tokens
  11. SSE citations event (doc_id, chunk_id, page, scores, preview)
  12. Postgres rag.audit_log + Langfuse trace
```

Anti-hallucination discipline: if no reranked chunk supports the answer, the system responds in the user's language with an explicit "not found" line. There is no creative-mode toggle.

4.4 Storage layout

```
${DATA_ROOT:-/var/lib/reineke}/
├─ postgres/          main app DB (rag schema)
├─ authentik-db/      identity DB
├─ langfuse-db/       tracing DB
├─ redis/             AOF
├─ minio/             raw/{doc_id}/... + export/
├─ qdrant/            vectors + snapshots
├─ duckdb/            reineke.duckdb (single file)
├─ ollama/            model weights (~80 GB if all tiers)
├─ tei/              reranker cache
├─ docling/          OCR models cache
├─ loki/             logs
└─ grafana/          dashboards + state
```

All dashboards are provisioned JSON; no admin clicks required to stand them up. All mounts are per-service subdirectories under one root, which keeps the backup/restore script readable.

5. Data model (PostgreSQL — schema rag)

Seven tables capture every concern that is not a vector or a blob:

- `rag.users` — mirror of Authentik sub/email/groups, refreshed on every JWT validation.
- `rag.folders` — logical folder tree; `acl_groups TEXT[]` is authoritative.
- `rag.documents` — every ingested file (incl. `sha256`, `status`, `minio_key`, `pages`).

- `rag.chunks` — admin visibility mirror; vectors live in Qdrant.
- `rag.tables` — one row per DuckDB-registered table (XLSX sheet or PDF table).
- `rag.audit_log` — every retrieval-api query: user, class, retrieved doc ids, SQL (if any), LLM, tokens, latency, answer hash, Langfuse trace ref.
- `rag.jobs` — RQ mirror for the admin UI (ephemeral state becomes persistent).

See [docs/02_ARCHITECTURE.md §4](#) for the DDL.

6. Security and ACL model

Four pillars:

1. **Identity:** Authentik (OIDC, RS256 / 2048-bit, access token 15 min, refresh 24 h). Groups are the unit of authorisation.
2. **Trust boundary:** Caddy is the only host-exposed surface. Service-to-service uses either bearer JWT (Authentik) or a shared `INTERNAL_SERVICE_TOKEN`.
3. **ACL enforcement:** `acl_groups` is an indexed payload field on every Qdrant point. Every search carries a mandatory `payload.acl_groups ANY user.groups` filter. DuckDB exposes per-group-hash views; the `duckdb-api` parses the generated SQL, rejects anything but `SELECT`, and executes only against the ACL view.
4. **Audit:** every query (hashed answer included) lands in `rag.audit_log`. Authentik login events go to Loki. GDPR export is `rag-admin audit export --format csv`.

Escape hatch: a long-lived, randomly-rotated `ADMIN_BACKUP_TOKEN` grants admin access when Authentik itself is down. Logged loudly when used.

7. LLM routing (ADR-004)

A YAML router config consumed by `retrieval-api`:

```
classes:
  lookup:      { model: gemma2:9b-instruct-q5_K_M,  max_tokens: 400 }
  extraction:  { model: qwen2.5:32b-instruct-q4_K_M, max_tokens: 1200 }
  table-math:  { model: qwen2.5:32b-instruct-q4_K_M, max_tokens: 800 }
  synthesis:   { model: llama3.3:70b-instruct-q4_K_M, max_tokens: 1600 }
  embedding:   { model: bge-m3, dimensions: 1024 }
  reranker:    { model: BAAI/bge-reranker-v2-m3, server: tei }
```

Resource profile on the M4 Max 64 GB reference box:

Model	Quant	RAM approx	Role	Avg wall-time first token
Gemma 2 9B	Q5_K_M	7.5 GB	fast path, classifier	≤ 2 s
Qwen 2.5 32B	Q4_K_M	20 GB	reasoning, extraction, table-math	≤ 6 s
Llama 3.3 70B	Q4_K_M	40 GB	synthesis (rarely loaded)	≤ 20 s
bge-m3	–	1 GB	dense + sparse embed	–
bge-reranker-v2-m3	–	0.6 GB	reranker (TEI)	p95 ≤ 500 ms / 12 candidates

OLLAMA_MAX_LOADED_MODELS=2 on Apple Silicon keeps Metal from thrashing. A LLM_PROFILE=compact fallback drops the heavy tier entirely.

8. Observability

Three separate signals, kept separate on purpose:

- **Langfuse** — one parent span per query with children for `classify`, `rewrite`, `dense_search`, `sparse_search`, `rerank`, `sql_plan`, `sql_exec`, `generate`. Model name, inputs, outputs, latency, token counts attached. Primary debugging surface.
- **Prometheus + Grafana** — `rag_query_total{class}`, `rag_query_latency_seconds{phase,class}`, `rag_retrieval_hits{source}`, `rag_ingestion_jobs_total{state}`, plus container/host metrics. Four provisioned dashboards: Overview / Ingestion / Infra / Quality.
- **Loki + Promtail** — stdout of every container. 14-day INFO retention, 90-day WARN+.

Every custom service exposes `/metrics` (Prometheus format) and `/healthz`, and publishes a `rag_build_info{version,commit}` gauge.

9. Build model: coordinator + seven specialists

The repo is designed to be **built by AI agents**, not typed in by a single engineer. Eight agent definitions live in `.claude/agents/`, each with a tight lane:

Agent	Phase(s)	Owns (in repo)
coordinator	all	BUILD_LOG.md , HANDOVER.md ; dispatches, verifies, never writes app code
deployment-agent	1, 9	docker-compose.yml , Makefile , .env.example , Caddyfile, backup/restore scripts
auth-agent	2	Authentik blueprints, services/common/auth.py (JWT lib)
llm-agent	3	scripts/pull-models.sh , scripts/smoke-*.sh , config/retrieval/models.yaml
ingestion-agent	4, 5	services/docling/** , services/ingestion-api/** , services/duckdb-api/** , migrations/**
retrieval-agent	6, 8	services/retrieval-api/** , config/retrieval/prompts/** , scripts/eval.py , regression tests
ui-agent	7	config/openwebui/** , config/pipelines/reineke_rag.py
observability-agent	9	config/langfuse/** , config/prometheus/** , config/grafana/provisioning/** , config/loki/**

Each phase has **explicit acceptance criteria** (A1.1 through A10.1) that the coordinator verifies by running scripts — never by trusting a subagent's self-report. On failure, it re-dispatches the owning subagent with the failure log. Three strikes escalate to the human owner.

See [docs/05_IMPLEMENTATION_PLAN.md](#) for the phase gates and [docs/06_AGENT_BRIEFS.md](#) for per-agent contracts.

Rough budget: ~18 engineer-days of equivalent work; 3–5 wall-clock days for an agent build if the owner is responsive on Phase 0 and Phase 8 loops.

10. Architecture Decision Records

Eight accepted ADRs freeze the significant choices:

#	Title	Key outcome
ADR-001	Docling for document parsing	One parser, structure preserved, HybridChunker default
ADR-002	Qdrant for vector + sparse	Native hybrid (1.10+), indexed payload filters, int8 quant
ADR-003	bge-m3 as the single embedder	Dense + sparse from one model, 1024-d, DE+EN equal
ADR-004	Tiered Ollama + router	9B / 32B / 70B routed by query class
ADR-005	Authentik as IdP	OIDC + groups, blueprint-reproducible
ADR-006	DuckDB SQL path	Numeric answers <i>computed</i> , not generated
ADR-007	Open WebUI + custom pipeline	One "Reineke-RAG" model entry; built-in upload disabled
ADR-008	Thin FastAPI over LangChain/LlamaIndex	Components yes, frameworks no

Rule of thumb: if it isn't in an ADR or architecture doc, it is implementation detail and a subagent may choose freely within its lane.

11. Why the n8n self-hosted AI starter kit was rejected

The n8n starter kit is a useful *demo*. Reineke-RAG exists because it breaks on real office corpora for four structural reasons ([concept §1](#)):

1. **Flat-text parsing** destroys table structure in PDFs and XLSX.
2. **Fixed-size chunking** splits mid-table and separates headings from bodies.
3. **Dense-only retrieval** misses exact terms (part numbers, product codes, German compounds).
4. **One LLM for every query** — no path that actually computes a sum in a spreadsheet, no synthesis-grade model for cross-doc work.

Reineke-RAG's design reverses each: structure-aware parsing (Docling) → structure-aware chunking (HybridChunker) → hybrid retrieval + rerank (Qdrant + TEI) → tiered LLM routing with a real SQL branch for tables.

It additionally adds what the starter kit lacks: **citations, folder ACLs, audit log, offline contract, bilingual prompts, observability per query**.

12. Operational numbers to carry in your head

Dimension	Target
Corpus supported	500 – 10 000 docs (growing)
Ingestion throughput	100 mixed docs < 20 min on M4 Max
Retrieval quality gate	recall@3 ≥ 85 %, recall@10 ≥ 95 % on 50-query gold set
Citation fidelity	100 % (zero hallucinated citations allowed)
lookup latency	p95 ≤ 5 s to last token
synthesis latency	p95 ≤ 60 s
Qdrant sizing (1 M chunks, int8)	~10 GB disk, ~4 GB RAM
Ollama model ceiling	~80 GB (all tiers + embed + rerank)
Backup cadence	nightly; GFS 7 daily / 4 weekly / 12 monthly
Admin onboarding from handbook only	< 30 min

13. Forward compatibility

Documented extension points so v1.x additions don't force a rewrite:

- **New mime types** → add a parser branch in Docling service (PPTX, HTML, MD already near-free).
- **New embedder** → new Qdrant collection (vector shape changes), blue/green reindex.
- **New ACL predicates** (per-doc tags, confidentiality level) → add payload fields + filter clauses; migration backfills.
- **New data sources** (wiki crawl, mail) → normalise to a DoclingDocument-like JSON; submit to `ingestion-api`.
- **Two-host split** (v1.1) when corpus > 5 k docs and team > 30 users — documented path: worker host (Ollama + TEI + ingestion-worker) + app host (everything else) over a Docker overlay or wireguard.

Backwards compatibility is explicit in the versioning rules: an embedder change or a chunking change that alters boundaries is a breaking (MAJOR) bump; prompt changes and LLM swaps are MINOR/PATCH.

14. Summary

Reineke-RAG is a **deliberately boring** RAG stack — boring in the complimentary, Dan McKinley sense. Every hot component has a proven counterpart in production somewhere: Postgres, Redis, MinIO, Qdrant, Docker, OIDC. The *interesting* choices are concentrated where they buy measurable quality: Docling for structure-preserving parsing, bge-m3 for dense+sparse in one model, tiered LLM routing, a real SQL path for tables, a shared reranker. Everything wraps around a single offline contract and a mandatory ACL filter.

The repository today is the blueprint. The stack is instantiated by an agent build that the repo itself orchestrates.

For the current repository state, read in order:

1. [README.md](#) — orientation
2. [docs/01_CONCEPT.md](#) — the "why"
3. [docs/02_ARCHITECTURE.md](#) — the "how"
4. [docs/adr/](#) — the "why not X"
5. [docs/05_IMPLEMENTATION_PLAN.md](#) — the build schedule

To use the finished stack → see [USER_HANDBOOK.md](#). **To run the finished stack** → see [TECHNICAL_HANDBOOK.md](#).